

دليل تعليمي مختصر وعملي

جمع متطلبات العميل وبناء وكيل محادثة

من كلام العميل إلى ملفات واضحة وسلوك ذكي قابل للاختبار

ستتعلم فقط ما تحتاجه:

- كيف تسأل الشركة قبل كتابة الكود
- كيف تحول كل خدمة إلى حقول مطلوبة
- كيف تقسم الوكيل إلى ملفات ومسؤوليات
- كيف يتصرف مع الترحيب والإساءة والغموض والتكرار

1

افهم نوعين من المتطلبات

لا تخطئ إعداد الوكيل بمتطلبات زبون الشركة

أولاً: متطلبات الشركة

تجمعها مرة قبل البناء: خدمات الشركة، حدود الإجابات، سياسات السعر، الأقسام، متى نحول لموظف، وما البيانات المطلوبة لكل خدمة.

ثانياً: متطلبات العميل النهائي

يجمعها الوكيل أثناء المحادثة. مثال: نوع الموقع، هدفه، المزايا، اللغات، وهل يوجد موقع حالي.

إذا لم تفصل بين المستويين ستضع أسئلة العملاء داخل الـ Prompt، وسيصبح تغيير خدمة واحدة مؤثراً في كامل الوكيل.

المسار الصحيح

المرحلة	النتائج
مقابلة الشركة	وثيقة خدمات وسياسات وحدود
تعريف الكتلوج	حقوق مطلوبة لكل خدمة
المحادثة	حقائق مستخرجة مع دليل من كلام العميل
المراجعة	ملخص أو مسودة يؤكد بها العميل

2

مقابلة الشركة قبل البرمجة

الأسئلة التي تمنع التخمين لاحقا
اجلس مع صاحب العمل أو القسم المختص، واكتب الإجابات بصيغة قابلة للتحويل إلى بيانات.

ما الذي نسجله	مثال للسؤال	اسأل عن
مفتاح واسم ووصف كل خدمة	ما الخدمات التي يستطيع العميل طلبها؟	الخدمات
حقوق مطلوبة واختيارية	ما المعلومات اللازمة قبل قبول الطلب؟	المتطلبات
أسعار، مواعيد، وعود أو قرارات ممنوعة	ما الذي لا يجوز للوكيل تأكيده؟	الحدود
أسباب التحويل والقسم المسؤول	متى يجب تدخل موظف؟	التحويل
نبرة وقواعد ثابتة	كيف تريدون الرد على الإساءة والغموض؟	الردود
مؤشرات واختبارات قبول	كيف نعرف أن الوكيل نجح؟	النجاح

لا تقبل عبارة: نريد وكيلا ذكيا

حولها إلى أفعال قابلة للاختبار: يصنف الرسالة، يحدد الخدمة، يجمع الحقوق، يشرح الغموض، يعرض ملخصا، ثم يطلب تأكيدا.

3

حول الخدمات إلى كتالوج متطلبات

الكتالوج هو مصدر الأسئلة، وليس ال Prompt لكل خدمة أنشئ تعريفا يحتوي الاسم والوصف والقسم وقائمة الحقول. كل حقل يحتاج مفتاحا ثابتا، اسما عربيا، سؤالاً، نوع قيمة، وهل هو مطلوب.

مثال مبسط

```
ServiceDefinition(
    key="website_development",
    department="txsaas",
    requirements=[
        Requirement(key="business_type", question="ام؟كطاشن لاجم ام"),
        Requirement(key="website_goal", question="ام؟عقوملا فده ام"),
        Requirement(key="existing_website", kind="boolean"),
        Requirement(key="features", kind="text"),
        Requirement(key="languages", kind="human_languages"),
    ],
)
```

- لا تكتب سؤال المتطلب في workflow.py؛ اقرأه من service_catalog.py.
- استخدم مفاتيح ثابتة بالإنجليزية، واعرض عناوين عربية للمستخدم.
- حدد نوع القيمة لمنع خلط لغة الواجهة بلغة البرمجة أو نعم/لا بنص حر.
- عند إضافة خدمة جديدة عدل الكتالوج واختباراته، لا منطق الحوار كله.

4

تقسيم المجلدات

كل ملف يملك قرارا واحدا واضحا
هيكل مقترح

```
src/agent/  
  domain/  
    message_decision.py  
    conversation_state.py  
    service_catalog.py  
    policy_decision.py  
  application/  
    message_classifier.py  
    decision_guard.py  
    requirements_collector.py  
    policy_engine.py  
    response_builder.py  
  prompts/  
    message_classifier.py  
  graph/  
    workflow.py  
  api/  
    schemas.py  
    routes.py  
  tests/  
    test_decision_guard.py  
    test_requirements_collector.py  
    test_dialogue_behavior.py
```

قاعدة التقسيم

إذا وجدت ملفا يصنف الرسالة ويتحقق منها ويجمع المتطلبات ويكتب الرد، فهو كبير أكثر من اللازم. افصل كل قرار حتى تختبره منفردا.

5

ملفات Domain

العقود والحالة التي يفهمها كل المشروع

يحتوي	وظيفته	الملف
intent, service, requirements, evidence, confidence	شكل قرار المصنف	message_decision.py
الخدمة، الحقول، السؤال الأخير، العدادات، المرحلة	ذاكرة الحوار الحالية	conversation_state.py
الخدمات والمتطلبات والأسئلة والأنواع والأقسام	تعريف خدمات الشركة	service_catalog.py
continue, explain, handoff, block	شكل قرار السياسة	policy_decision.py

ماذا لا تضع هنا؟

لا تضع اتصال نموذج، قاعدة بيانات، API أو نصوص ردود. Domain يصف البيانات والقواعد الأساسية فقط.

أهم حقول ConversationState

- الخدمة التي يجمع الوكيل متطلباتها: current_service
- ما جمع وما بقي: collected_requirements و missing_requirements
- حتى يفهم أن «داخل الموقع» جواب لسؤال قناة التشغيل: last_question_key
- لفهم التكرار والسياق: last_user_message و last_assistant_response
- عدادات منفصلة، لا تخط بينهما: clarification_attempts و semantic_repeat_count

6

ملفات Application

من يقترح ومن يتحقق ومن يقرر

الملف	السؤال الذي يجب عنه	ممنوع عليه
message_classifier.py	ماذا يقصد العميل وما الحقائق المذكورة؟	تخزين الحقائق أو صياغة الرد النهائي
decision_guard.py	هل قرار النموذج مدعوم بكلام العميل؟	اختراع معنى جديد أو استدعاء النموذج
requirements_collector.py	أي حقول نخزن وما السؤال التالي؟	تقرير أسلوب الرد أو سياسة الإساءة
policy_engine.py	ما التصرف المسموح الآن؟	كتابة رد طويل أو تعديل المتطلبات
response_builder.py	كيف نشرح القرار للعميل؟	تغيير القرار أو اختراع بيانات

التسلسل الذهبي: النموذج يقترح الفهم ← Guard ينظفه ← Collector يحدث الحالة ← Policy يقرر التصرف ← Response Builder يصوغ الرد.

المصنف و Decision Guard

اجعل النموذج مساعد فهم لا مصدر حقيقة

المصنف يستخرج

- نوع الرسالة: ترحيب، طلب خدمة، سؤال، توضيح، إساءة، خارج النطاق، تأكيد أو تعديل.
- الخدمة المرشحة مع confidence.
- كل متطلب صريح مع value و evidence من رسالة العميل نفسها.

ال Guard يتحقق

- يحذف أي متطلب لا يملك evidence موجودا حرفيا في الرسالة.
- لا يسمح بإعادة ادعاء سؤال سعر من رسالة سابقة.
- لا يسمح بتغيير خدمة نشطة بسبب كلمة عابرة ضعيفة.
- يربط الجواب القصير بالسؤال النشط؛ مثل «داخل الموقع» = deployment_channel.
- في مرحلة مراجعة المسودة يحول طلب الإضافة الواضح إلى ticket_edit.

لماذا ملف Guard مستقل؟

لأن ال Prompt نص توجيهي وقد يخطئ النموذج. Guard كود حتمي يمكن اختباره، وهو المكان الصحيح لتصحيح الادعاءات غير الموثوقة.

السلوكيات الأساسية للوكيل

ما الذي يجب أن يفعله في الرسائل الشائعة؟

نوع الرسالة	السلوك الصحيح	مثال مختصر
ترحيب	يرحب ويعرض المساعدة دون بدء متطلبات	مرحباً، كيف أساعدك؟
إساءة	يهدئ ولا يرد بإساءة ثم يعيد الحوار للمهمة	سأساعدك باحترام؛ ما الخدمة المطلوبة؟
خارج النطاق	يوضح نطاقه ويقترح خدمات الشركة	أنا مخصص لخدمات الشركة التقنية.
غامض	يسأل سؤال اختيار بسيط بدل قائمة طويلة	هل تقصد موقعاً أم تطبيقاً أم استضافة؟
لم أفهم	يعيد شرح السؤال بمثال واقعي	ما العمل الذي سيقوم به الوكيل بدل الموظف؟
سؤال جانبي	يجيب باختصار ثم يعود للسؤال المعلق	أنا مساعد الشركة. نعود: أين سيعمل الوكيل؟
طلب موظف	يحول بصدق إذا كانت القناة موجودة	سأحول الطلب مع ملخص ما جمعناه.
سعر	يطبق سياسة الشركة ولا يخترع رقماً	يحدد القسم السعر بعد مراجعة المتطلبات.

التوضيح والتكرار

ليسا الشيء نفسه

الحالة	كيف تكتشفها	ماذا تفعل
تكرار حرفي	النص نفسه بعد التطبيق	غير الصياغة، ولا تطلب تحققاً بشرياً فوراً
تكرار معنوي	نفس الهدف بصيغ مختلفة	اختصر وأعط خيارات أو مثالا
فشل توضيح	العميل يقول لم أفهم جواباً لسؤالك	بسط السؤال نفسه واحفظ last_question_key
إرسال سريع	طلبات كثيرة خلال نافذة زمنية	منفصل عن ذكاء الحوار Rate limit

الخطأ الشائع

لا تجعل semantic_repeat_count سبباً تلقائياً لاتهام العميل أو طلب CAPTCHA. قد يكون العميل محتاراً. التحقق البشري يخص سلوكاً آلياً أو إساءة استخدام واضحة، أما الحيرة فتحتاج شرحاً أفضل.

- المرة الأولى: أسأل السؤال الطبيعي.
- المرة الثانية: أعدّه بكلمات أبسط مع مثالين أو ثلاثة.
- المرة الثالثة: اعرض اختيارات قصيرة أو اقترح موطفاً إن كان متاحاً.
- عندما يجيب العميل، صفر clarification_attempts وانتقل للحقل التالي.

كيف يجمع Collector المتطلبات

احفظ الحقيقة ثم اسأل عن أول نقص

- اختر الخدمة من أقوى مرشح، أو احتفظ بالخدمة الحالية عند جواب متابعة.
- طبع أسماء الحقول aliases إلى المفاتيح الرسمية في الكتالوج.
- لا تخزن إلا ما اجتاز confidence و Decision Guard.
- احسب required fields الناقصة بترتيب الكتالوج.
- احفظ last_question_key و last_question_text قبل إرسال السؤال.
- عند اكتمال الحقول انتقل إلى review واعرض ملخصاً للتأكيد.

منطق مبسط

```
service = choose_service(state, decision)
facts = guard(decision, current_message)

for fact in facts:
    key = normalize_key(service, fact.key)
    state.collected_requirements[key] = fact.value

missing = required_fields(service) - collected_fields(state)
state.last_question_key = first(missing)
state.stage = "requirements" if missing else "review"
```

11

رتب الملفات ولا تخلطها: Workflow

مسار واحد واضح لكل رسالة

الترتيب	المكون	الناتج
1	Classifier	قرار أولي منظم
2	Decision Guard	قرار موثوق بالحاضر
3	Policy Engine	سماح، توضيح، تحويل أو منع
4	Requirements Collector	حالة محدثة وسؤال نال
5	Response Builder	رد عربي مناسب للسياق
6	State Store	حفظ الحالة للجولة القادمة

المقاطعة الصحيحة

إذا سأل العميل «ما اسمك؟» أثناء جمع المتطلبات، أجب ثم احتفظ بالسؤال المعلق. في الرسالة التالية يستكمل الوكيل من `last_question_key` بدل بدء المحادثة من جديد.

ينسق هذه الخطوات فقط. لا تضع داخله نصوص السياسات أو تفاصيل كل خدمة `workflow.py`.

12

متى تستخدم LangChain ومتى LangGraph؟

اختر أقل طبقة تحقق الحاجة

الخيار	استخدمه عندما	قوته الأساسية
النموذج مباشرة SDK	لديك استدعاء واحد: تصنيف أو استخراج أو إجابة	أقل كود وأقل تعقيد
LangChain	تريد ربط نموذج و Prompt و Structured Output و Tools أو Retriever	عقود موحدة وتكاملات و Agent loop و Middleware
LangGraph	لديك مراحل وحالة وفروع واضحة أو توقف واستئناف	تنسيق stateful مع persistence و HITL و durable execution
LangChain + LangGraph	تريد مكونات LangChain داخل عقد Workflow مضبوط	مرونة النموذج مع تحكم صريح في المسار

في وكيل جمع المتطلبات

استخدم LangChain لبناء المصنف والمخرجات المنظمة واستدعاء الأدوات. استخدم LangGraph لأن الحوار يمر بمراحل discovery ثم requirements ثم review، وله حالة وفروع تأكيد وتعديل.

- لا تستخدم LangGraph لمسألة سؤال وجواب واحدة بلا حالة أو فروع.
- لا تجعل Agent loop يقرر خطوات يجب أن تكون ثابتة مثل: لا تنشئ التذكرة قبل التأكيد.
- ضع القرارات الحرة عند النموذج، والقرارات التجارية الثابتة في Nodes و Edges حتمية.
- استخدم LangSmith عند الحاجة إلى تتبع المسارات وتقييم الردود، لا كجزء من منطق العمل.

Tools وSkills وRAG وSubgraphs

أسماء متشابهة لكن وظائف مختلفة

المكون	متى تستخدمه	مثال
Tool	عندما يجب تنفيذ فعل أو جلب بيانات حية بعقد واضح	إنشاء تذكرة، قراءة موعد، استعلام API
Skill	عندما يحتاج الوكيل تعليمات أو معرفة تخصصية عند الطلب	قواعد خدمة CYBTX أو طريقة إعداد عرض
RAG / Retriever	عندما تكون المعلومات كثيرة أو متغيرة وتحتاج مصادر	سياسات الشركة ووثائق الخدمات
Middleware	لسلوك يعبر كل الاستدعاءات	اختيار أدوات، Logging، retries، guardrails
Subgraph	لمسار فرعي له خطوات وحالة ويمكن إعادة استخدامه	تعديل مسودة أو حجز موعد

Tool ليست Skill

تنفذ فعلا حقيقيا. إذا كانت الخاصية «كيف أتصرف؟» فهي Tool تعطي الوكيل تعليمات وسياقا متخصصا عند الحاجة؛ Skill. Tool، وإذا كانت «نفذ أو اجلب» فهي Skill.

لماذا Skills؟

لأنها Progressive Disclosure: يرى الوكيل وصفا صغيرا، ويحمل التعليمات الكاملة فقط عند الحاجة. هذا يقلل ازدحام ال Prompt واستهلاك السياق.

Router أم Supervisor أم Handoff؟

لا تبدأ بأثقل نمط

النمط	متى يصلح	متى لا تحتاجه
Router	تصنيف واحد يرسل الطلب إلى تخصص واحد	إذا كان نفس الوكيل يستطيع معالجة كل الأنواع
Skills	وكيل واحد يحتاج تخصصات نصية عند الطلب	إذا كان التخصص يملك حالة وأدوات مستقلة كثيرة
Subagents + Supervisor	عدة خبراء يتعاونون في مهمة متعددة الخطوات	إذا كان المطلوب مجرد اختيار قسم واحد
Handoff	ينتقل التحكم لمختص يتحدث مباشرة مع المستخدم	إذا كان المنسق يجب أن يبقى الواجهة الوحيدة
Subgraph	مسار مستقل لكن مضبوط وحتمي نسبياً	إذا كانت خطوة واحدة أو دالة بسيطة

هل Travel-X يحتاج Supervisor الآن؟

ليس بالضرورة. إذا كان الطلب يذهب إلى قسم واحد فقط، يكفي Router أو Registry مع Department Agent. استخدم Supervisor فقط عندما يحتاج الطلب نفسه إلى أكثر من خبير بالتتابع أو بالتوازي، ثم يحتاج جمع نتائجهم.

سلم القرار الأبسط

- استخراج واحد؟ استخدم Structured Output.
- فعل خارجي؟ أضف Tool.
- تعليمات تخصصية عند الطلب؟ أضف Skill.
- مراحل وحالة وفروع؟ أضف LangGraph.
- اختيار تخصص واحد؟ أضف Router.
- عدة متخصصين يتعاونون على المهمة نفسها؟ عندها فقط فكر في Supervisor.

أين تضع هذه المكونات؟

الاسم المعماري يتحول إلى ملف واضح

مسؤوليته	الملف المقترح	الحاجة
العقد والفروع وترتيب التنفيذ	graph/workflow.py	الرئيسي LangGraph
لقسم أو عملية مستقلة Subgraph	graph/department_subgraph.py	مسار تخصصي
التعليمات والمراجع ومتى تحمل	skills/<name>/SKILL.md أو JSON	تعريف Skill
اكتشافها والتحقق منها وإتاحتها	infrastructure/skill_registry.py	تحميل Skills
واجهة الفعل دون ربط بالتنفيذ	application/ports/<tool>.py	Tool contract
تنفيذ API أو قاعدة البيانات	infrastructure/<tool>_adapter.py	Tool adapter
اختيار قسم أو مسار واحد	application/router.py	Router
تنسيق عدة Agents عند وجود مبرر	application/supervisor.py	Supervisor

لا تنشئ supervisor.py لمجرد أن لديك أقساما متعددة. أنشئه فقط بعد اختبار يثبت أن مهمة واحدة تحتاج أكثر من Agent وأن Router واحد لا يكفي.

مثال كامل واختبارات القبول

من عبارة العميل إلى متطلبات مؤكدة

الرد المتوقع	ما يحدث داخليا	الرسالة
ما العمل الذي تريد أن يقوم به؟	ناقص service=ai_agent, agent_goal	أريد وكبلا
مثال: الرد على العملاء أو حجز المواعيد.	clarification_attempts += 1	لم أفهم
أين سيعمل: موقع أم تطبيق؟	حفظ agent_actions و agent_goal	للرد وحجز المواعيد
أنا مساعد الشركة. نعود: أين سيعمل؟	لا حذف للسؤال النشط, intent=identity	ما اسمك؟
ما البيانات التي سيعتمد عليها؟	ربطه ب deployment_channel	داخل الموقع

اختبارات لا تسلم الوكيل بدونها

- الترحيب لا يبدأ تذكرة ولا يملأ متطلبات.
- الإساءة لا تغير الخدمة ولا تنتج ردا مسيئا.
- خارج النطاق لا يجعل الوكيل يخترع إجابة.
- جواب المتابعة القصير يملأ الحقل الصحيح.
- سؤال جانبي لا يمحو السؤال المعلق.
- المتطلب بلا evidence لا يخزن.
- تكرار المعنى يغير الشرح ولا يطلب تحققا بشريا بلا سبب.
- اكتمال الحقول ينتج ملخصا مطابقا لما قاله العميل.

الخلاصة: ابدأ بالكتالوج، اجعل النموذج يقترح، وال Guard يتحقق، وال Collector يخزن، وال يشرح. بهذه الحدود تستطيع تغيير الشركة أو نوع Response Builder يقرر، وال Policy الوكيل دون إعادة بناء كل شيء.

مراجع رسمية مختصرة: docs.langchain.com/oss/python/langchain/overview —

docs.langchain.com/oss/python/langgraph/overview — docs.langchain.com/oss/python/langchain/multi-agent